

砚台：详细设计文档

文档版本: v2.0

编写日期: 2026-06-18

适用范围: 网络小说智能生成平台（砚台）

文档状态: 已整理

1. 引言

1.1 编写目的

本文档为「砚台」网络小说智能生成平台的系统详细设计文档（System Detailed Design, SDD），在《需求分析文档》基础上，对系统架构、模块划分、数据模型、接口契约、关键算法、AI 编排机制、安全方案与部署运维进行详细阐述，为开发实现、测试验证与运维部署提供技术蓝图。

1.2 设计目标

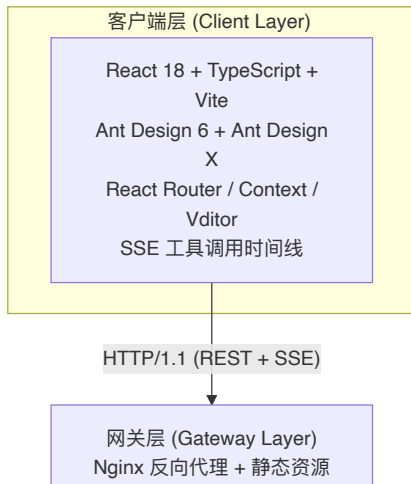
- 高内聚低耦合**：按业务领域划分前后端模块，AI 编排层与业务服务层解耦。
- 项目级隔离**：任何数据访问与 AI 工具调用必须以 `project_id` 为边界，杜绝跨项目污染。
- 流式实时性**：AI 生成过程通过 SSE 实时推送到前端，提供沉浸式交互体验。
- 可观测与可恢复**：利用 LangGraph Checkpoint 实现会话状态持久化，支持异常中断后的无缝恢复。
- 渐进式扩展**：LLM 供应商、子 Agent 类型、技能模板、知识实体类型均支持插件化扩展。
- 创作者友好**：内容正文以 Markdown 文件存储，便于创作者直接读写与版本控制。

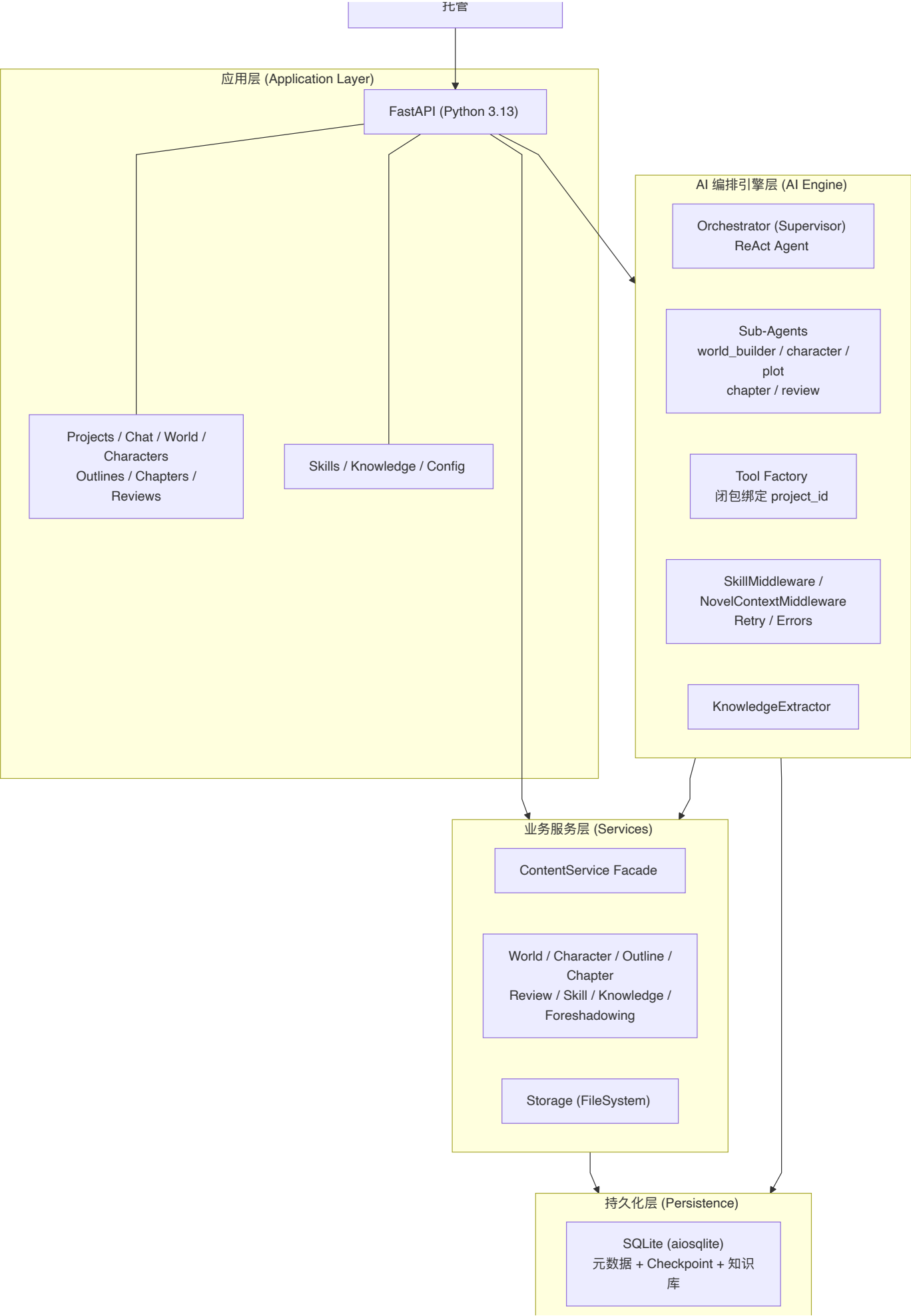
1.3 设计约束

- LLM 供应商锁定**：当前仅支持 Anthropic Claude，API 调用兼容 `langchain-anthropic`。
- 同步工具约束**：LangGraph 工具函数为同步签名，内部异步 I/O 通过 `asyncio.run()` 桥接。
- 内容存储分离**：SQLite 仅存储元数据与索引，实际创作内容必须存储为 Markdown 文件。
- 项目隔离约束**：所有子 Agent 的工具与上下文必须通过 `project_id` 闭包绑定，禁止模块级单例。

2. 系统架构设计

2.1 总体架构图



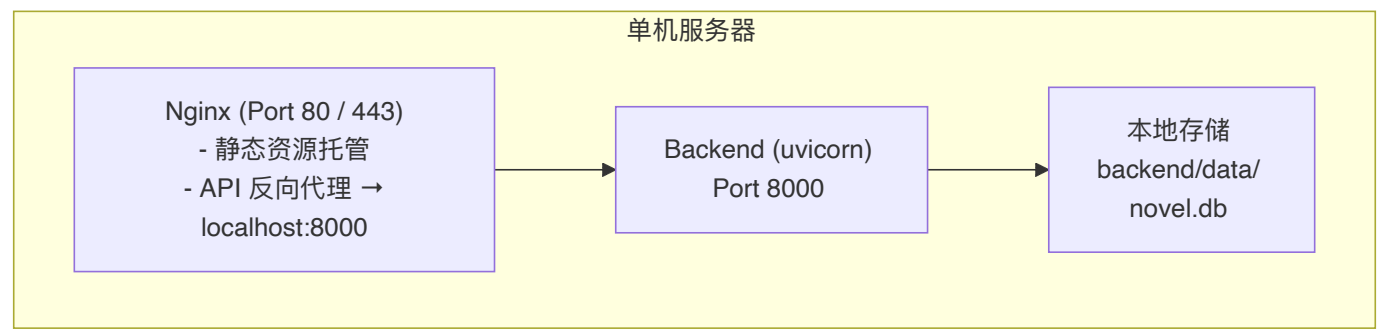




2.2 技术栈矩阵

层级	技术选型	版本/说明
前端框架	React	18.3+
前端语言	TypeScript	5.5+
构建工具	Vite	5.4+
UI 组件库	Ant Design + Ant Design X	6.x + 2.x
前端路由	React Router DOM	6.26+
前端状态	React Context	无 Redux/Zustand
Markdown 编辑	Vditor	3.11+
Markdown 渲染	marked + DOMPurify + @ant-design/x-markdown	—
后端框架	FastAPI	0.115+
后端语言	Python	3.13+
ASGI 服务器	Uvicorn	0.34+
配置管理	Pydantic Settings	—
包管理器	uv（后端）、npm（前端）	—
LLM 框架	LangChain + LangGraph	—
LLM 供应商	Anthropic (Claude)	当前唯一支持
语义嵌入	sentence-transformers	all-MiniLM-L6-v2
数据库	SQLite +aiosqlite	WAL 模式
Checkpoint	langgraph-checkpoint-sqlite (AsyncSqliteSaver)	会话持久化
文件存储	FileSystemStorage（自研）	原子写入 + 路径安全
测试框架	pytest + pytest-asyncio + pytest-cov	asyncio_mode=auto
类型检查	mypy	strict=true
代码规范	ruff	line-length=120
反向代理	Nginx（可选）	单机部署

2.3 部署架构（单机部署）



- 前端： `npm run build` 生成 `dist/` 静态产物，由 Nginx 直接托管。
- 后端：Uvicorn 监听 8000 端口，通过 `systemd` 或 `supervisor` 管理。
- 数据：`novel.db` 与 `backend/data/` 存储在本地磁盘，单文件即可备份。
- 开发模式：前端 `vite dev` (port 5173) 通过 proxy 转发 `/api` 到 `localhost:8000`。
- Docker： `docker compose up` 同时启动前端（:3000）与后端（:8001）。

3. 前端模块设计

3.1 目录结构

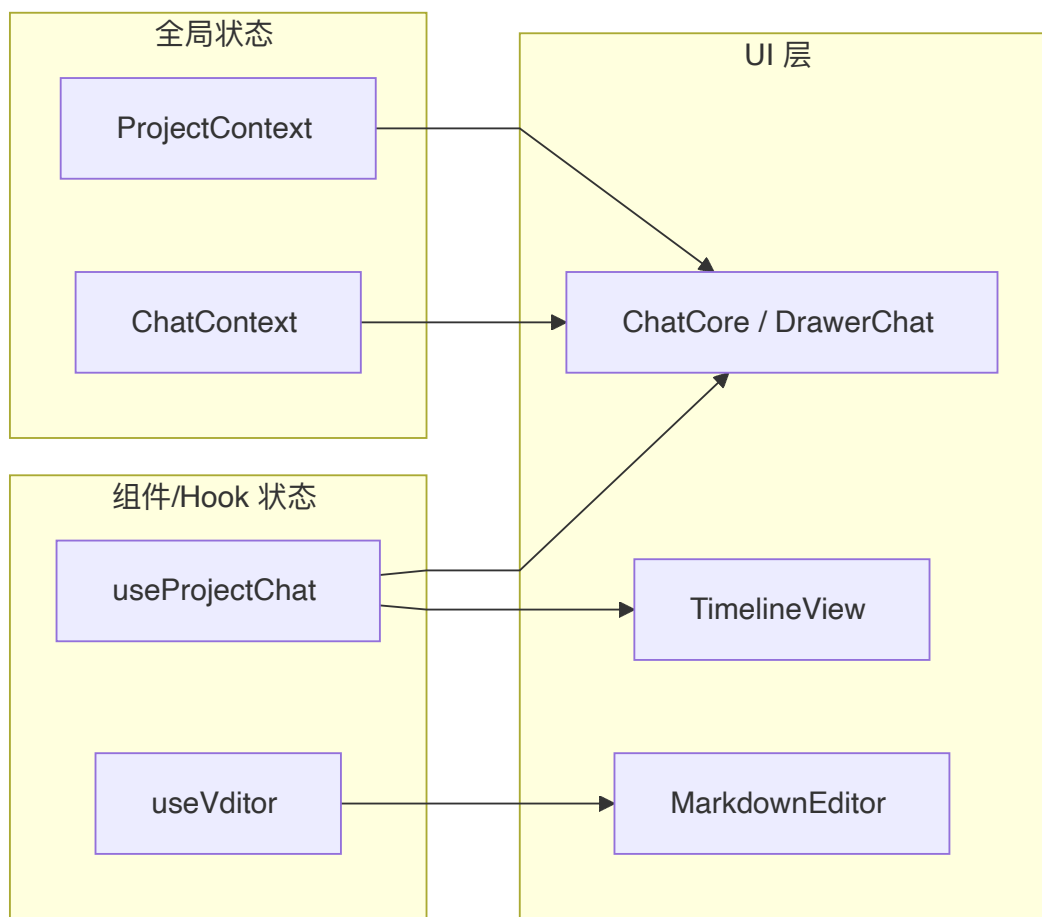
```
frontend/src/
├─ main.tsx                # React 应用入口
├─ App.tsx                 # 路由与主题配置
├─ api/                    # Axios API 客户端
│  ├─ client.ts            # baseUrl / 拦截器
│  ├─ projects.ts
│  ├─ sessions.ts
│  ├─ world.ts
│  ├─ characters.ts
│  ├─ outlines.ts
│  ├─ chapters.ts
│  ├─ reviews.ts
│  ├─ skills.ts            # 技能 API
│  ├─ knowledge.ts         # 知识库 API
│  └─ config.ts            # 系统配置 API
├─ contexts/               # React Context
│  ├─ ProjectContext.tsx
│  └─ ChatContext.tsx
├─ hooks/                  # 自定义 Hooks
│  ├─ useProjectChat.ts    # SSE 连接与消息解析
│  └─ useVditor.ts
├─ pages/                  # 页面级组件
│  ├─ Dashboard.tsx
│  ├─ world/
│  │  ├─ characters/
│  │  ├─ outline/
│  │  ├─ chapters/
│  │  └─ chat/
│  ├─ settings/
│  └─ skills/              # SkillManager / SkillEdit
└─ components/
```

├─ layout/	# AppLayout / IconSidebar / SecondaryNav
├─ ai/	# ChatCore / DrawerChat / WelcomeScreen
├─ dashboard/	
├─ cards/	
├─ common/	# MarkdownEditor / PageHeader / EmptyState
└─ knowledge/	# KnowledgeConfirmPanel
├─ types/	
└─ index.ts	
├─ utils/	
└─ format.ts	
└─ frontmatter.ts	
└─ styles/	
└─ theme.css	# 设计 Token + 程序化纹理
└─ vditor-override.css	# 编辑器稿纸质感

3.2 路由与布局矩阵

路径	页面组件	布局	上下文依赖
/	Dashboard	DashboardLayout	—
/projects/:id/world	WorldList	AppLayout	ProjectContext
/projects/:id/world/:worldId	WorldEdit	AppLayout	ProjectContext
/projects/:id/characters	CharacterList	AppLayout	ProjectContext
/projects/:id/characters/:charId	CharacterEdit	AppLayout	ProjectContext
/projects/:id/outline	OutlineEditor	AppLayout	ProjectContext
/projects/:id/chapters	ChapterList	AppLayout	ProjectContext
/projects/:id/chapters/:chapterId	ChapterEdit	AppLayout	ProjectContext
/projects/:id/chat[:sessionId]	ChatPage	AppLayout	ProjectContext + ChatContext
/projects/:id/settings	ProjectSettings	AppLayout	ProjectContext
/projects/:id/knowledge	KnowledgeBase	AppLayout	ProjectContext
/settings	SystemSettings	SystemLayout	—
/skills	SkillManager	SystemLayout	—
/skills/new	SkillEdit	SystemLayout	—
/skills/:skillName/edit	SkillEdit	SystemLayout	—

3.3 前端状态流



3.4 关键组件设计

A. ChatCore 与 SSE 处理

- `useProjectChat.ts` 管理原生 `fetch` + `ReadableStream` SSE 连接。
- 消息状态机: `idle` → `connecting` → `streaming` → `complete` | `error`。
- 事件路由:
 - `message` → 追加到当前 assistant 消息
 - `tool_start` / `tool_end` → 构建时间线节点
 - `thinking` → 折叠展示推理过程
 - `error` → 显示错误横幅
 - `done` → 标记完成, 触发知识增量检查
 - `knowledge_conflicts` → 提示潜在冲突

B. OutlineEditor

- 基于 Ant Design `Tree`, 启用 `draggable` 属性。
- 节点数据映射到 `outlines_meta` 表, `parent_id` 表达层级, `sort_order` 表达同层顺序。
- 选中节点后右侧加载对应 Markdown 文件内容。

C. MarkdownEditor

- 对 Vditor 的 React 封装, 默认 `mode: 'sv'` (分屏编辑 + 实时预览)。
- 通过 `gray-matter` 解析/生成 YAML Frontmatter。

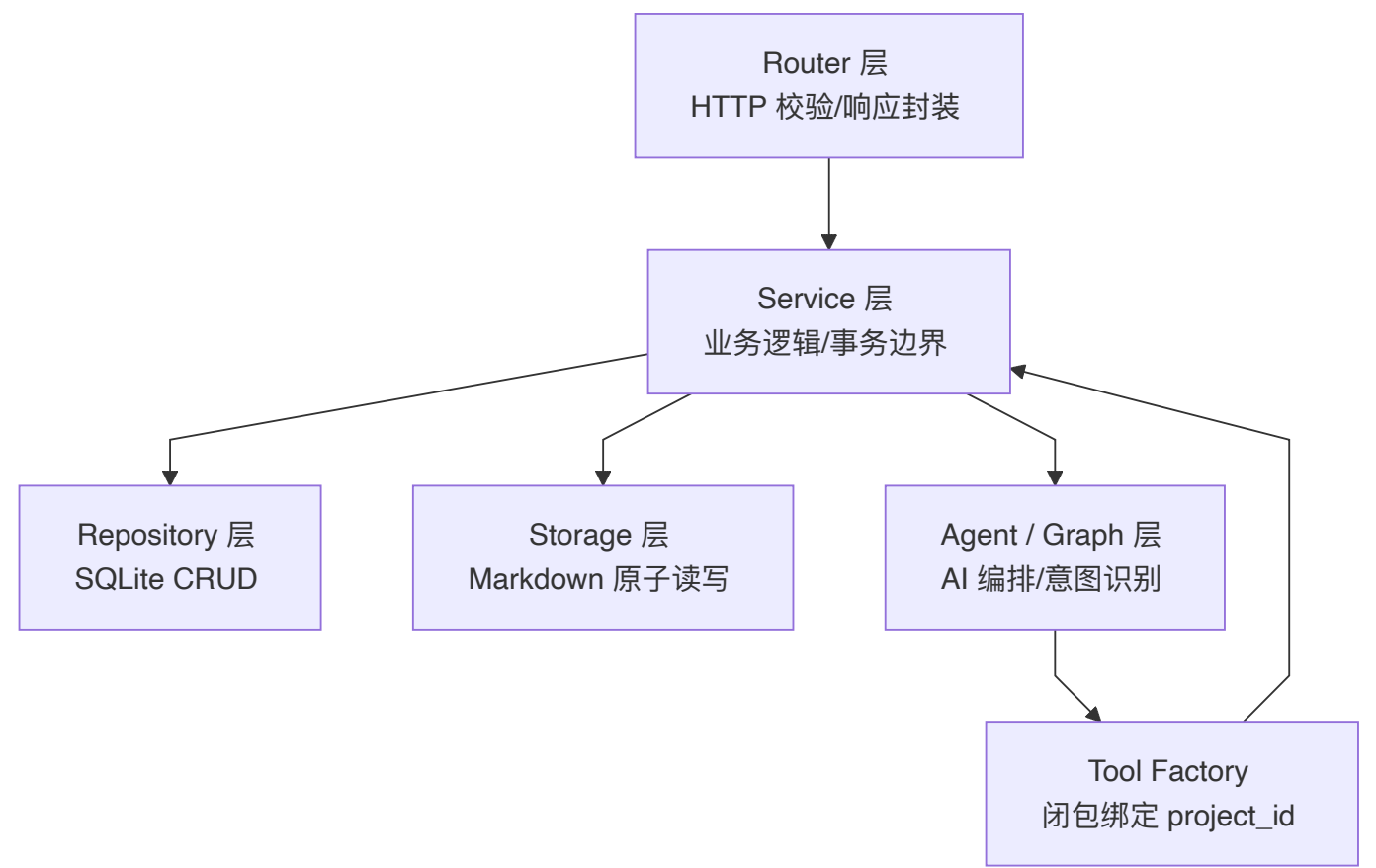
- 编辑区使用 `.paper-ruled` 稿纸纹理，光标改为朱红色。

4. 后端模块设计

4.1 目录结构

```
backend/app/
├── main.py                # FastAPI 工厂函数与 lifespan
├── config.py              # Pydantic Settings
├── api/v1/                # REST API 路由
│   ├── projects.py
│   ├── chat.py            # SSE 流式端点
│   ├── world.py
│   ├── characters.py
│   ├── outlines.py
│   ├── chapters.py
│   ├── reviews.py
│   ├── skills.py
│   ├── knowledge.py       # 知识库 API
│   └── config.py         # 系统配置 API
├── core/
│   ├── agent/
│   │   ├── langchain_subagents.py # 5 个子 Agent 工厂
│   │   ├── tool_factory.py        # 项目级工具工厂
│   │   ├── skill_tools.py         # 运行时创建技能工具
│   │   ├── skillmiddleware.py     # 运行时 skill 注入
│   │   └── novel_contextmiddleware.py
│   ├── graph/
│   │   ├── builder.py             # Orchestrator 图编译
│   │   └── state.py               # OrchestratorState / ProjectContext
│   ├── prompts/                  # 系统提示词模板
│   ├── errors.py                 # AIError / classify_llm_error
│   ├── retry.py                  # with_retry 指数退避
│   ├── logging_utils.py          # 结构化 JSON 日志
│   └── serialization_utils.py    # safe_json_value
├── db/
│   ├── connection.py             # aiosqlite 连接管理
│   ├── schema.sql                # 数据库 Schema
│   └── checkpoint.py             # AsyncSqliteSaver
├── services/                     # 业务服务层
│   ├── content.py                # ContentService Facade
│   ├── base.py                   # BaseContentService
│   ├── storage.py                # FileSystemStorage
│   ├── llm.py                    # create_llm_client 工厂
│   ├── world.py
│   ├── character.py
│   ├── outline.py
│   ├── chapter.py
│   ├── review.py
│   ├── skill.py                  # Skill 解析/CRUD
│   ├── knowledge.py              # KnowledgeService
│   ├── knowledge_extractor.py    # LLM 知识提取
│   ├── foreshadowing.py         # 伏笔管理
│   ├── session_repository.py
│   ├── project_repository.py
│   └── config_repository.py      # 全局配置
```

4.2 分层职责与调用规则



层级	职责	调用规则
Router	HTTP 请求校验、参数解析、响应封装	仅调用 Service，禁止直接访问 DB/文件
Service	业务逻辑编排、事务边界、数据转换	可调用 Repo/Storage/LLM，禁止直接引用 Agent
Repository	数据库增删改查（原始 SQL + aiosqlite）	仅被 Service 调用
Storage	Markdown 文件原子读写	仅被 Service 调用，内部做路径安全校验
Agent/Graph	AI 编排、意图识别、子 Agent 调度	通过 Tool 调用 Service 完成写操作
Tool Factory	生成项目级工具（闭包绑定 <code>project_id</code> ）	被 Agent 层持有

4.3 AI 编排引擎详细设计

A. Orchestrator (Supervisor)

- 实现文件： `backend/app/core/graph/builder.py`
- 采用 `langchain.agents.create_agent` 构建顶层 ReAct Agent 图。
- 系统提示词要求 Orchestrator LLM 将用户意图分类为 5 类之一：
 - `world_builder`：世界观、地理、文化、力量体系
 - `character`：角色档案、关系网、性格弧光

- `plot`: 情节、大纲、章节目录
- `chapter`: 正文撰写、润色、续写
- `review`: 质量审阅、一致性检查
- Handoff 通过返回 `Command(goto="<agent_name>", update={...})` 实现显式路由。

B. Sub-Agents

- 实现文件: `backend/app/core/agent/langchain_subagents.py`
- 每个子 Agent 是独立编译的 LangGraph 图, 包含:
 1. 领域专属系统提示词 (`backend/app/core/prompts/tools/`)
 2. 项目级工具集 (`tool_factory.create_*_tools(project_id)`)
 3. `SkillMiddleware`: 运行时注入匹配 domain 的 Skill

Agent	Domain	核心工具	技能过滤
world_builder	world	create/edit/delete world, query/get content	<code>world</code>
character	character	create/edit/delete character, query/get content	<code>character</code>
plot	plot	create/edit/delete outline, get_outline_tree	<code>plot</code> , <code>outline</code>
chapter	chapter	create/edit/delete chapter, query/get content	<code>chapter</code>
review	review	create review, query/get content	<code>review</code>

C. 工具工厂与隔离机制

- 实现文件: `backend/app/core/agent/tool_factory.py`
- 关键约束: 工具绝不能为模块级单例。`create_*_tools(project_id)` 返回通过 Python 闭包绑定 `project_id` 的 `@tool` 函数。
- 写工具示例: `create_world_setting(project_id, name, content)` → 内部调用 `WorldSettingService.create(...)`。
- 读工具: `query_content`、`get_content`、`get_outline_tree` 为通用查询工具。

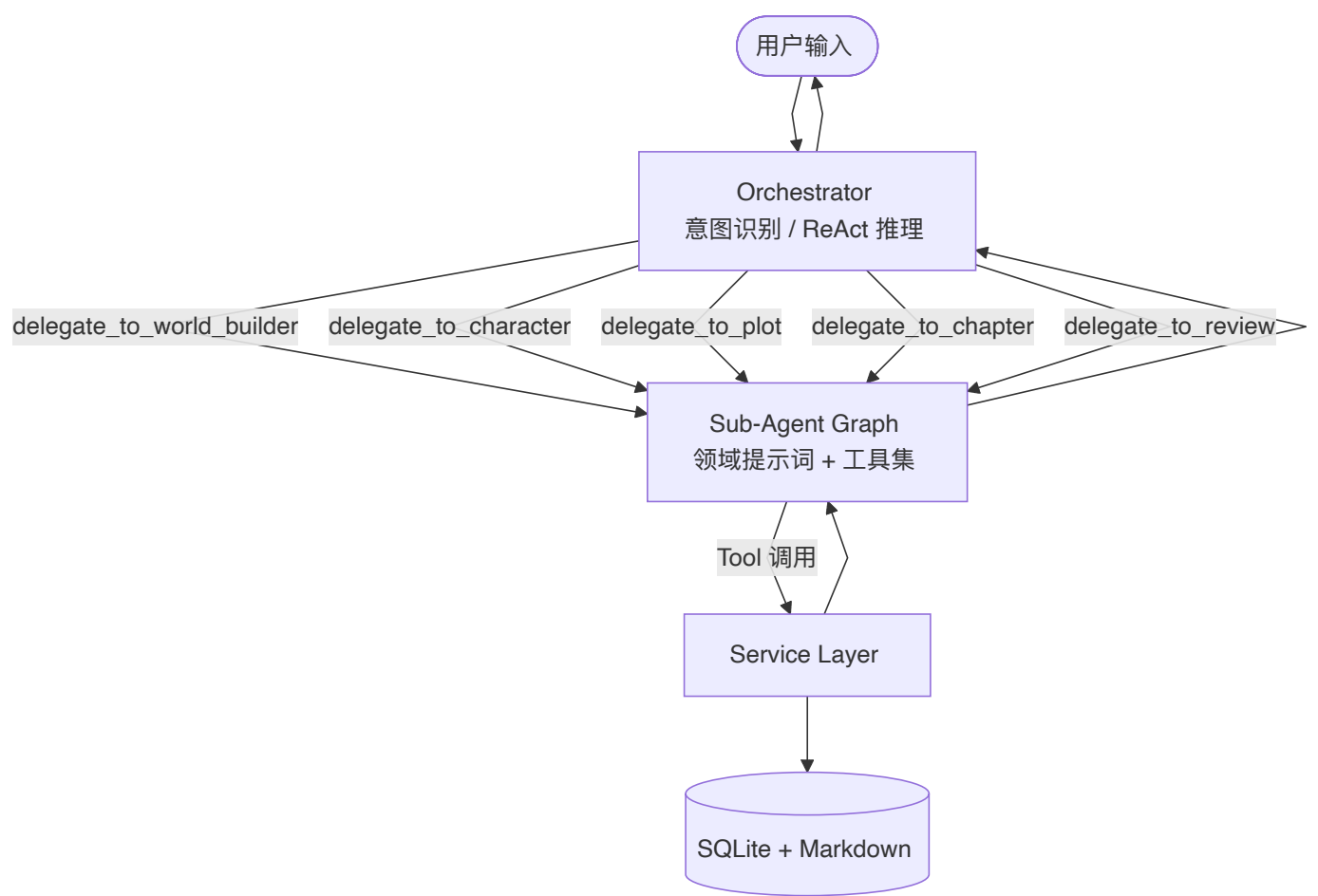
D. State Schema

- 实现文件: `backend/app/core/graph/state.py`

```
class ProjectContext(TypedDict):
    project_name: str
    project_description: str | None
    world_settings: list[dict[str, str]]
    characters: list[dict[str, str]]
    outline: dict[str, str] | None
    chapters: list[dict[str, str]]
    knowledge_summary: dict | None    # 跨会话知识摘要

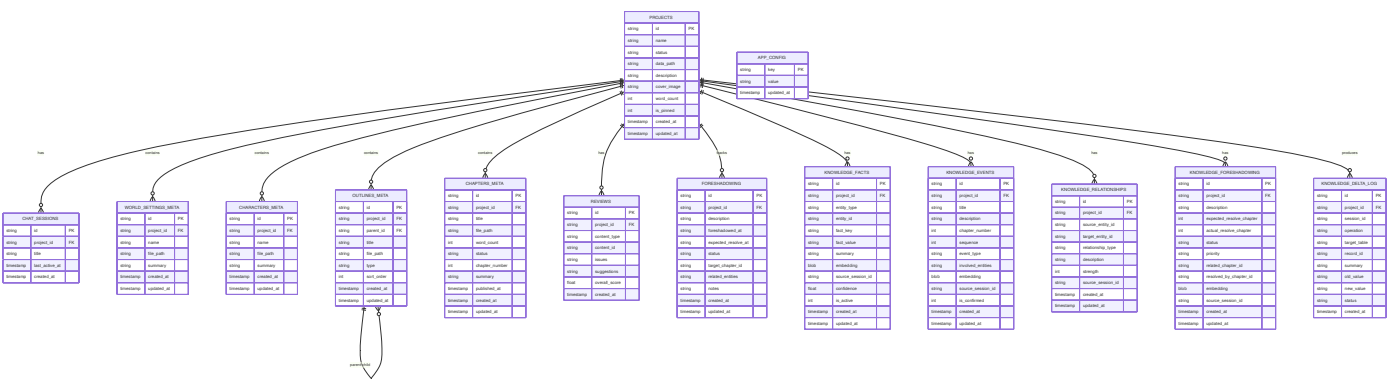
class OrchestratorState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]
    session_id: str
    project_id: str
    project_context: ProjectContext
```

4.4 多智能体协作流程



5. 数据模型设计

5.1 E-R 关系图（逻辑模型）



5.2 混合存储策略

数据类型	存储介质	理由
项目/会话/元数据	SQLite (<code>novel.db</code>)	结构化查询、事务、轻量
世界观正文	<code>data/{pid}/world/{id}.md</code>	大文本、版本控制友好
角色正文	<code>data/{pid}/characters/{id}.md</code>	同上
大纲节点正文	<code>data/{pid}/outlines/{id}.md</code>	同上
章节正文	<code>data/{pid}/chapters/{id}.md</code>	同上
审阅记录	SQLite (<code>reviews</code>)	结构化评分与问题列表
技能模板	<code>data/skills/{name}.md</code>	创作者可手动编辑、Git 友好
结构化知识	SQLite + 向量嵌入	语义检索、跨会话共享
LangGraph Checkpoint	SQLite (共享 <code>novel.db</code>)	状态恢复、单文件备份
全局配置	SQLite (<code>app_config</code>)	键值对配置

5.3 文件存储安全设计

- `FileSystemStorage._resolve(project_id, relative_path)` 严格校验解析后的绝对路径位于 `DATA_DIR/{project_id}/` 之内。
- 写入采用原子操作：先写入临时文件，再 `os.rename()` 覆盖目标文件。
- 读取使用 `aiofiles` 异步 IO，避免阻塞事件循环。

6. 接口设计

6.1 REST API 概览

所有业务 API 以 `/api/v1` 为前缀。内容资源统一采用 `POST /{id}/update` 与 `POST /{id}/delete`（非 PATCH/DELETE），保持风格一致。

资源	端点	方法	说明
Health	<code>/health</code>	GET	健康检查
Projects	<code>/projects</code>	POST / GET	创建 / 列出
	<code>/projects/{id}</code>	GET / PATCH / DELETE	详情 / 更新 / 删除
Chat Sessions	<code>/projects/{pid}/sessions</code>	POST / GET	创建 / 列出
	<code>/projects/{pid}/sessions/{sid}</code>	DELETE	删除会话
	<code>/projects/{pid}/sessions/{sid}/history</code>	GET	会话历史
Chat Stream	<code>/projects/{pid}/sessions/{sid}/chat/stream</code>	POST	SSE 流式对话
World	<code>/projects/{pid}/world</code>	POST / GET	创建 / 列出
	<code>/projects/{pid}/world/{wid}</code>	GET	详情
	<code>/projects/{pid}/world/{wid}/update</code>	POST	更新

	/projects/{pid}/world/{wid}/delete	POST	删除
Characters	/projects/{pid}/characters	POST / GET	创建 / 列出
	/projects/{pid}/characters/{cid}	GET	详情
	/projects/{pid}/characters/{cid}/update	POST	更新
	/projects/{pid}/characters/{cid}/delete	POST	删除
Outlines	/projects/{pid}/outlines	POST / GET	创建 / 列出
	/projects/{pid}/outlines/{oid}	GET	详情
	/projects/{pid}/outlines/{oid}/update	POST	更新
	/projects/{pid}/outlines/{oid}/delete	POST	删除
Chapters	/projects/{pid}/chapters	POST / GET	创建 / 列出
	/projects/{pid}/chapters/{cid}	GET	详情
	/projects/{pid}/chapters/{cid}/update	POST	更新
	/projects/{pid}/chapters/{cid}/delete	POST	删除
Reviews	/projects/{pid}/reviews	POST / GET	创建 / 列出
	/projects/{pid}/reviews/{rid}	GET	详情
Skills	/skills	GET / POST	列出 / 创建
	/skills/{name}	GET	详情
	/skills/{name}/update	POST	更新
	/skills/{name}/delete	POST	删除
Knowledge	/projects/{pid}/knowledge/summary	GET	知识摘要
	/projects/{pid}/knowledge/timeline	GET	时间线
	/projects/{pid}/knowledge/facts	GET	事实
	/projects/{pid}/knowledge/relationships	GET	关系
	/projects/{pid}/knowledge/foreshadowing	GET	伏笔
	/projects/{pid}/knowledge/deltas	GET	待确认增量
	/projects/{pid}/knowledge/deltas/confirm	POST	确认增量
	/projects/{pid}/knowledge/deltas/reject	POST	驳回增量
	/projects/{pid}/knowledge/search	POST	语义搜索
Config	/config	GET / POST	读取 / 更新全局配置

6.2 SSE 流式对话接口

端点： `POST /api/v1/projects/{project_id}/sessions/{session_id}/chat/stream`

请求体：

```
{
  "message": "帮我生成主角的详细性格设定",
  "options": {
    "temperature": 0.7
  }
}
```

响应: Content-Type: text/event-stream

Event Type	说明	示例数据结构
message	LLM 文本流 token	<code>{"type": "message", "content": "他性格", "delta": true, "source": "orchestrator"}</code>
tool_start	工具开始执行	<code>{"type": "tool_start", "tool": "delegate_to_character", "input": {}}</code>
tool_end	工具执行结束	<code>{"type": "tool_end", "tool": "delegate_to_character", "output": "..."} </code>
thinking	推理内容流	<code>{"type": "thinking", "content": "..."} </code>
knowledge_conflicts	知识冲突提示	<code>{"type": "knowledge_conflicts", "conflicts": [...]}</code>
error	错误	<code>{"type": "error", "code": "RATE_LIMIT", "message": "..."} </code>
done	流程结束	<code>{"type": "done", "session_id": "...", "full_content": "..."} </code>

状态码:

- 200 : SSE 流正常建立
- 403 : Session 不属于该 Project
- 404 : Session 不存在
- 502/503/504 : LLM 服务不可用或超时

6.3 核心数据模型

ProjectBase

```
{
  "id": "string",
  "name": "string",
  "status": "active",
  "description": "string | null",
  "cover_image": "string | null",
  "word_count": 0,
  "is_pinned": 0,
  "created_at": "ISO8601",
  "updated_at": "ISO8601"
}
```

ChatMessage

```
{
  "role": "user | assistant | tool",
  "content": "string",
  "tool_calls": [],
  "timestamp": "ISO8601"
}
```

ReviewResult

```
{
  "id": "string",
  "content_type": "world | character | outline | chapter",
  "content_id": "string",
  "issues": "string (Markdown)",
  "suggestions": "string (Markdown)",
  "overall_score": "float (0.0 ~ 10.0)",
  "created_at": "ISO8601"
}
```

SkillTemplate

```
---
name: "黄金三章写作法"
domain: ["chapter", "plot"]
description: "指导开篇三章的钩子设计"
tags: ["beginner", "hook"]
---
# 提示词正文...
```

7. 关键算法与流程

7.1 项目上下文加载流程

为避免 Orchestrator 与 Sub-Agent 的上下文窗口被海量正文撑爆，系统采用元数据摘要注入策略：

1. **启动时**：builder.py 根据 project_id 加载摘要：
 - 项目基本信息
 - 世界观列表（仅名称 + 摘要）
 - 角色列表（仅名称 + 身份摘要）
 - 大纲树结构（仅标题层级）
 - 章节列表（仅标题 + 状态）
 - 知识摘要（已确认的关键事实、事件、关系、伏笔）
2. **Agent 需要详情时**：通过 get_content / query_content 工具按需读取完整 Markdown。
3. **状态更新**：Sub-Agent 执行写操作后，同步更新 ProjectContext 中对应摘要条目。

7.2 流式事件处理流程（v2 Streaming）

后端采用 LangGraph v2 streaming API：

```

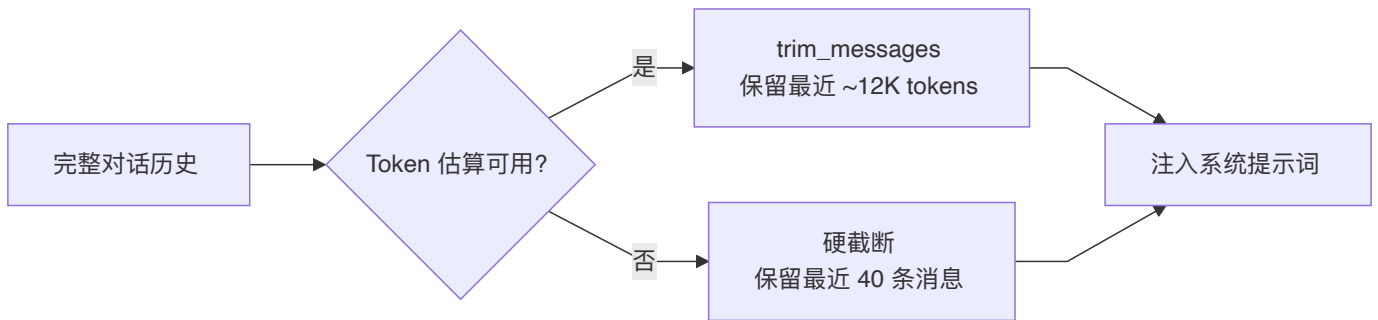
async for event in graph.astream(
    input_state,
    config,
    stream_mode=["messages", "updates", "custom"],
    subgraphs=True,
):
    # 事件分类:
    # - on_chat_model_stream → 文本/thinking token
    # - on_tool_start / on_tool_end → 工具调用时间线
    # - on_chain_end → 子图完成
    yield f"event: message\ndata: {json.dumps(payload)}\n\n"

```

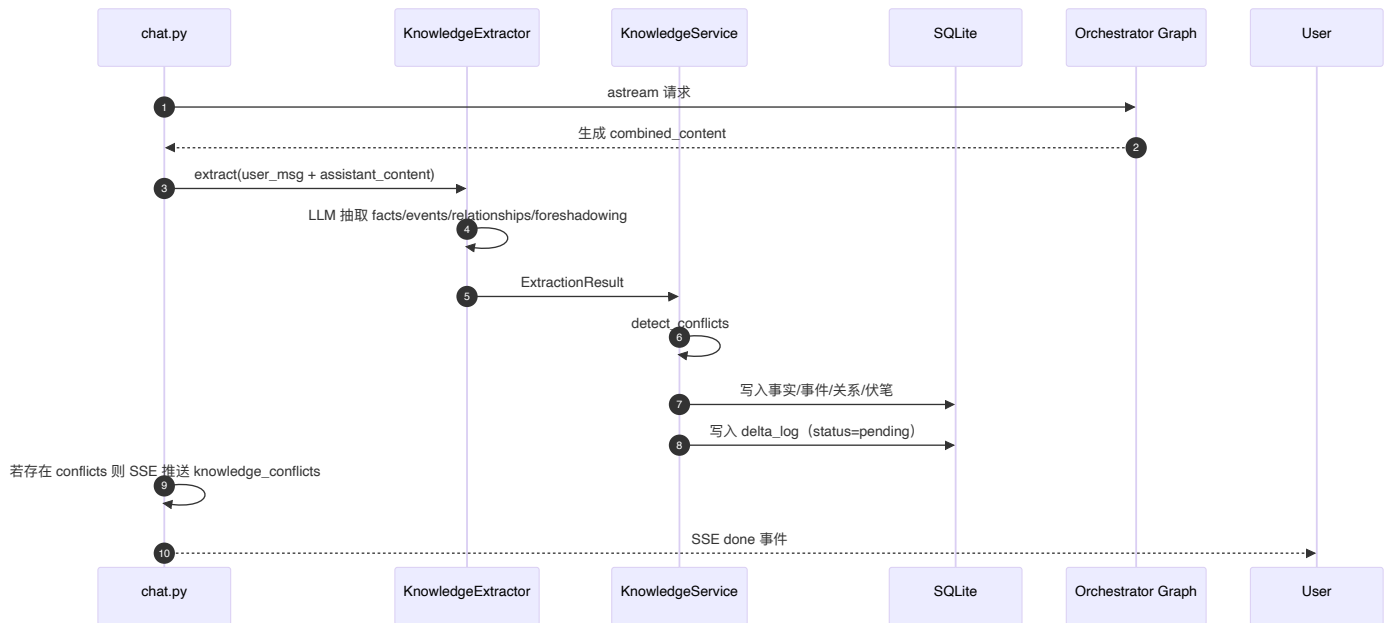
前端 `useProjectChat.ts` 维护 `EventParser`:

1. 接收 SSE chunk, 按 `\n\n` 分割为事件帧。
2. 根据 `event` 类型路由到对应处理器, 更新 React 状态。
3. 子 Agent token 通过 `source != "orchestrator"` 判断, 路由到对应 delegate 工具的 subTimeline。

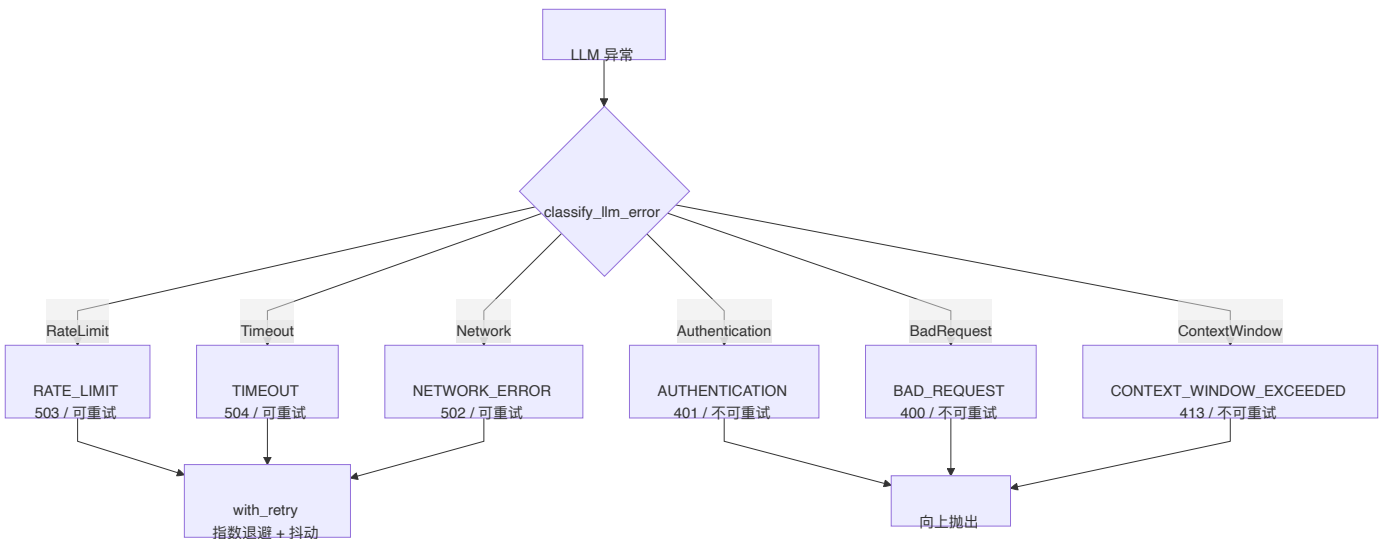
7.3 历史消息截断策略



7.4 知识提取与注入流程



7.5 错误分类与重试



8. 安全设计

8.1 访问控制

威胁	防护措施
跨项目数据访问	Router 层校验资源 <code>project_id</code> 与 URL <code>project_id</code> 一致性；Session 校验 <code>verify_project_binding()</code>
路径遍历攻击	<code>FileSystemStorage._resolve()</code> 严格校验绝对路径前缀；禁止 <code>..</code> 与绝对路径注入
LLM API Key 泄露	仅通过环境变量 <code>ANTHROPIC_API_KEY</code> 注入，不落入日志与数据库
CORS 滥用	显式配置 <code>allow_origins</code> ，生产环境仅允许已知域名
XSS	Markdown 渲染使用 <code>DOMPurify</code> 过滤；不直接执行用户输入

8.2 输入校验

- FastAPI 依赖 Pydantic 模型进行请求体验证。
- 文件名/路径中的非法字符在 Storage 层过滤或转义。
- 知识提取的 `fact_value` 等字段在落库前做长度与格式校验。

8.3 错误处理与信息隐藏

- 生产环境不返回 Python 异常堆栈，仅返回标准化错误码与友好提示。
- `AIError` 映射表：

LLM 异常	错误码	HTTP 状态	是否重试
RateLimitError	RATE_LIMIT	503	是
APITimeoutError	TIMEOUT	504	是
APIConnectionError	NETWORK_ERROR	502	是
AuthenticationError	AUTHENTICATION	401	否
BadRequestError	BAD_REQUEST	400	否
ContextWindowExceeded	CONTEXT_WINDOW_EXCEEDED	413	否

9. 部署与运维设计

9.1 单机部署方案

环境准备

```
# Python 3.13+ 与 uv
curl -LsSf https://astral.sh/uv/install.sh | sh
# Node.js 20+ (前端构建)
# Nginx (可选)
```

后端部署

```
cd backend
uv sync
cp .env.example .env
uv run uvicorn app.main:app --host 0.0.0.0 --port 8000
```

前端部署

```
cd frontend
npm install
npm run build
# 将 dist/ 复制到 Nginx 托管目录
```

Nginx 配置 (SSE 长连接)

```
server {
    listen 80;
    server_name your-domain.com;

    location / {
        root /opt/IterationThree/frontend/dist;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

```
location /api/v1/projects/ {
    proxy_pass http://127.0.0.1:8000;
    proxy_buffering off;
    proxy_cache off;
    proxy_set_header Connection '';
    proxy_http_version 1.1;
    chunked_transfer_encoding off;
}
}
```

9.2 Docker 部署

```
docker compose up
```

- 前端服务: port 3000
- 后端服务: port 8001
- 使用 Nanjing University 镜像源加速 npm/pypi

9.3 监控与日志

- 结构化日志: AI 相关操作输出 JSON 日志, 包含 `timestamp`, `level`, `event_type`, `project_id`, `session_id`, `error_code`, `retry_count`。
- 健康探针: `/health` 返回 `{"status": "healthy"}`。
- 未来扩展: LangSmith 链路追踪、Prometheus/Grafana 性能监控。

9.4 备份与恢复

- SQLite: `cp novel.db novel.db.backup`。
- Markdown 文件: `backend/data/` 目录整体备份, 兼容 Git。
- 建议策略: 每日定时快照 `novel.db` + `data/` 到对象存储或异地磁盘。

10. 附录

10.1 核心文件索引

文件路径	说明
backend/app/main.py	FastAPI 应用工厂
backend/app/core/graph/builder.py	Orchestrator 图构建
backend/app/core/graph/state.py	状态定义
backend/app/core/agent/tool_factory.py	项目级工具工厂
backend/app/core/agent/langchain_subagents.py	子 Agent 工厂
backend/app/core/agent/skill_middleware.py	Skill 运行时注入
backend/app/services/content.py	ContentService Facade
backend/app/services/storage.py	文件存储协议与实现
backend/app/services/knowledge.py	知识库服务
backend/app/services/knowledge_extractor.py	LLM 知识提取
backend/app/db/schema.sql	数据库 Schema
frontend/src/App.tsx	前端路由与主题
frontend/src/hooks/useProjectChat.ts	SSE 聊天逻辑
frontend/src/pages/skills/SkillManager.tsx	技能管理
frontend/src/components/knowledge/KnowledgeConfirmPanel.tsx	增量确认面板

10.2 设计决策记录（ADR）

决策	选项	选中方案	理由
前端框架	Vue 3 / React	React 18 + TS	生态丰富，Ant Design X 专为 React AI 聊天设计
状态管理	Redux / Zustand / Context	React Context	项目规模适中，Context 足够且减少依赖
ORM	SQLAlchemy / 原始 SQL	原始 SQL + aiosqlite	模型简单，避免 ORM 开销；LangGraph checkpointer 也直接用 SQLite
内容存储	DB BLOB / 文件系统	Markdown 文件	创作者可直接读写，Git 友好，避免大字段拖慢 DB
LLM 供应商	OpenAI / Anthropic / 本地	Anthropic Claude	当前唯一支持，代码预留工厂扩展点
流式协议	WebSocket / SSE	SSE	单向推送足够，SSE 更易穿透防火墙与代理
知识嵌入	本地模型 / 外部 API	sentence-transformers	单机部署友好，无需额外 API 依赖
技能注入	启动时静态 / 运行时动态	运行时动态读取文件	新增技能无需重启服务

10.3 修订记录

版本	日期	修订内容	作者
v1.0	2026-05-10	初始版本《系统详细设计文档》	—
v2.0	2026-06-18	整合技能系统、跨对话知识共享、纹理升级、系统配置等新增模块；细化架构图、E-R 图、流程图；补充知识提取与注入流程	